Parth Kumar Sinha

Independent Researcher

Oxford, United Kingdom

sinhaparth555@gmail.com

ORCID 0009-0002-3120-9301

---

14 September 2026

The Editor-in-Chief
IEEE Transactions on Parallel and Distributed Systems

Re: Submission of "Portable, Seekable Random Streams for Parallel CPU Simulation" as a regular paper

Dear Editor-in-Chief,

I am submitting "Portable, Seekable Random Streams for Parallel CPU Simulation" for consideration as a regular paper in IEEE Transactions on Parallel and Distributed Systems.

The paper starts from a defect rather than a design. Checkpointing a `std::mt19937` and restoring it under a different C++ standard library is not reliable: the libraries disagree on the numeric base of the serialized digits and on the order of the 624 state words. Under the Microsoft standard library the restore fails outright. Under libc++ it succeeds and resumes at the wrong position, with no diagnostic, which is the case that worries me. A user report against XGBoost 3.3.0 shows the first failure happening in the field.

Storing a position instead of an implementation removes the problem by construction, and for a counter-based generator the position is a key and an integer. What has kept that substitution out of CPU codebases is a performance objection. My own proposal to make the serialized generator state of XGBoost portable was declined on the grounds that Philox runs at one tenth the speed of the Mersenne Twister. The paper quotes that review comment verbatim and discloses in a footnote that I submitted the proposal it declined. The evaluation exists to answer that sentence.

It does not reproduce. Across five processors, unspecialised scalar Philox costs between 1.10 and 1.65 times what `std::mt19937` costs per byte, not ten times. With the wide multiplies specialised and counters interleaved across SIMD lanes, the bulk path costs between 0.22 and 0.75 times `std::mt19937`. In parallel, aggregate cost per byte does not move with the thread count, and the first limit on a sixteen-core host is hyperthread co-location rather than memory bandwidth.

Three things about the evaluation may interest the reviewers. Instruction supply is excluded by measurement rather than by argument: co-location costs 59 percent more per byte while L1 instruction-cache misses fall by 36 percent. The flat scaling result is reproduced under a second, independent threading runtime, so it is a property of the generator and not of my worker pool. And every placement study runs behind a gate that first measures whether the host can place threads at all, because a virtual machine accepts every affinity mask while the hypervisor places vCPUs itself, which silently turns two arms into one measurement taken twice.

I have tried to be straightforward about what the measurements do not show. xoshiro256++ is faster than vphilox on four of the five machines, and on ARM the buffered engine interface is still slower than `std::mt19937` even though the NEON kernel beneath it is faster. Both are in the paper. Execution-port contention is reported as an inference from the instruction mix rather than as a measurement, since the counters that would test it directly are not available on the one host that can control placement.

Every number traces to an archived run. The source, the raw JSON and the script that regenerates every table and figure are at `https://github.com/sinhaparth5/vphilox`, the measured release is archived at doi:10.5281/zenodo.22103483, and continuous integration fails the build if a derived file no longer matches the data behind it.

The manuscript is original, is not under consideration elsewhere, and has not been published in whole or in part. I am an independent researcher with no funding to declare. Beyond the XGBoost disclosure above I have no competing interests.

Thank you for your time.

Yours sincerely,

Parth Kumar Sinha